

Student Management System

JDBC + PostgreSQL + Maven

Project Report & Developer Guide

Chetan Kailas Banait

Aspiring Java Developer

April 2026

1. Project Overview

The Student Management System is a console-driven Java application that performs full CRUD operations on three core entities — Student, Course, and Enrollment — backed by a PostgreSQL database. The system demonstrates industry-standard JDBC practices including batch processing, stored procedure invocation, and callable statements.

1.1 Project Description

Built using Core Java with a clean 3-layer architecture (Entity → DAO → Service → Main), the application connects to a PostgreSQL database via JDBC. Users interact through a role-based console menu system supporting Student, Admin, and System roles.

1.2 Objectives

- Understand and implement JDBC database connectivity in Java
- Perform full CRUD operations using PreparedStatement
- Implement batch processing for high-volume inserts
- Call PostgreSQL stored procedures and functions from Java
- Apply layered (DAO + Service) architecture for clean separation of concerns
- Practice database normalisation and relational schema design
- Manage the project using Git & GitHub

2. Technologies Used

Technology	Version	Purpose
Java (Core Java)	JDK 8+	Main application language
PostgreSQL	14+	Relational database backend
JDBC	Built-in Java	Database connectivity API
PostgreSQL JDBC Driver	42.7.7	Java to PostgreSQL bridge
Maven	3.x	Dependency & build management
Eclipse IDE	Any	Development environment

The PostgreSQL JDBC driver (version 42.7.7) is declared as a compile-scope dependency in pom.xml and resolved automatically by Maven. No manual JAR management is needed.

3. Project Structure

The project follows the standard Maven directory layout and is organised into four Java packages:

```
StudentManagmentSystem/
├── pom.xml                               ← Maven dependencies
└── src/main/java/com/Student/
    ├── Entity/                           ← POJOs (data models)
    │   ├── Student.java
    │   ├── Course.java
    │   └── Enrollments.java
    ├── Dao/                              ← SQL / JDBC logic
    │   ├── StudentDao.java
    │   ├── CourseDao.java
    │   ├── EnrollmentDao.java
    │   ├── AllInsertDao.java
    │   └── AddDetails.java
    ├── Service/                           ← Menus & business logic
    │   ├── StudentService.java
    │   ├── AdminServiceec.java
    │   └── SystemService.java
    └── Main/                              ← Entry point
        └── StudentDriver.java
```

4. Database Design

Three normalised tables are used in the PostgreSQL database named StudentManagmentSystem. The enrolment table acts as a junction table implementing a many-to-many relationship between students and courses.

4.1 student table

Column	Type	Constraint	Description
student_id	INT	PRIMARY KEY	Unique student identifier
student_name	VARCHAR		Full name of the student
student_email	VARCHAR		Email address
dob	VARCHAR		Date of birth

4.2 course table

Column	Type	Constraint	Description
course_id	INT	PRIMARY KEY	Unique course identifier
course_name	VARCHAR		Name of the course
credits	DOUBLE		Credit hours

4.3 enrolment table (Junction Table)

Column	Type	Constraint	Description
enrolment_id	INT	PRIMARY KEY	Unique enrollment identifier
student_id	INT	FK → student	References student table
course_id	INT	FK → course	References course table
grade	VARCHAR		Assigned grade (A / B / C)

Relationship: One Student → Many Enrolments; One Course → Many Enrolments. This forms a classic M:N relationship resolved through the enrolment junction table.

5. Application Architecture

The system is built using a 4-layer architecture. Each layer has a single responsibility, making the code easy to maintain and extend.

5.1 Main Layer — `com.Student.Main`

StudentDriver.java

Entry point of the application. Contains `main()`. Runs an infinite `while(true)` loop displaying the top-level menu and delegates to the appropriate Service class based on user input.

- Press 1 → `StudentService.Studentmenu()`
- Press 2 → `AdminService.adminMenu()`
- Press 3 → `SystemService.systemMenu()`
- Press 4 → Exit application

5.2 Service Layer — `com.Student.Service`

Service classes act as menu controllers. They display options, read user input from Scanner, and delegate to the correct DAO method. No SQL logic lives here.

StudentService.java (7-option Student Menu)

- Register Student — `batch insert via StudentDao.registedStudentBatch()`
- Update Student Profile — `StudentDao.updateStudent()`
- Enroll in a Course — `CourseDao.insertCourseBatch()`
- View all Courses — `CourseDao.selectCourse()`
- View all Students — `StudentDao.selectAllStudent()`
- View Grade — `StudentDao.getStudentGrade()` [calls DB function]
- Go Back

AdminServiceec.java (7-option Admin Menu)

- Manage Courses — `CourseDao.callmethod()` [add / update / delete sub-menu]
- View all Students — `StudentDao.selectAllStudent()`
- View all Courses — `CourseDao.selectCourse()`
- View all Enrollments — `EnrollmentDao.selectEnrollments()`
- Assign Grade — `StudentDao.assignGrade()` [CallableStatement]
- Insert All — `AddDetails.addingAll()` [stored procedure]
- Go Back

SystemService.java (System / Advanced Menu)

- Insert via Stored Procedure — `AllInsertDao.insertAllData()`
- Batch Execution for Course — `CourseDao.insertCourseBatch()`
- Return

5.3 DAO Layer — `com.Student.Dao`

All JDBC database interactions live here. Each class handles operations for one or more entities.

`StudentDao.java`

- `registeredStudentBatch()` — Collects n students from user input and inserts them all in one go using `PreparedStatement.addBatch()` + `executeBatch()`. Far more efficient than looping single INSERTs.
- `updateStudent()` — Updates `student_name`, `student_email`, and `dob` for a given `student_id` using an UPDATE prepared statement.
- `selectAllStudent()` — Fetches all rows from the student table using `ResultSet` and prints them to console.
- `assignGrade()` — Uses `CallableStatement` with `{ call assign_grade_fn(?, ?) }` to invoke a PostgreSQL stored function that updates the grade field.
- `getStudentGrade()` — Calls the PostgreSQL function `get_student_grade(?)` via `SELECT * FROM get_student_grade(?)` and displays the returned table of `student_id`, `student_name`, and `grade`.

`CourseDao.java`

- `insertCourseBatch()` — Batch insert for multiple courses. Uses `addBatch()` loop then `executeBatch()`.
- `selectCourse()` — Displays all course records.
- `updateCourse()` — Updates `course_name` and `credits` by `course_id`.
- `deleteCourse()` — Deletes a course record by ID.
- `callmethod()` — Internal sub-menu loop for course CRUD, used by Admin.

`EnrollmentDao.java`

- `insertEnrollmentBatch()` — Batch inserts enrollment records (`enrolment_id`, `student_id`, `course_id`, `grade`).
- `selectEnrollments()` — Fetches and displays all enrollment records.

`AllInsertDao.java` — Stored Procedure Caller

The most important DAO for demonstrating stored procedures. Collects all student + course + enrollment data from the user and calls the PostgreSQL stored procedure `insert_all_data(...)` via `PreparedStatement` with 9 bind parameters. This inserts into all three tables in a single transactional call.

```
PreparedStatement ps = con.prepareStatement("CALL  
insert_all_data(?,?,?,?,?,?,?,?,?)");
```

`AddDetails.java`

A lightweight wrapper menu around `AllInsertDao.insertAllData()`. Provides a simple 2-option loop (insert all data / exit), called from `AdminService` option 6.

5.4 Entity Layer — `com.Student.Entity`

Plain Java Objects (POJOs) that mirror database table columns. Contain only private fields plus generated getters and setters. Zero database logic.

- `Student.java` — fields: `id`, `name`, `email`, `dob`

- Course.java — fields: id, Course_name, Credit
- Enrollments.java — fields: Enrolment_ids, Grade

6. SQL Files Description

Three SQL files accompany the project. Each contains a SELECT query for data inspection plus the DDL for stored routines.

6.1 student.sql

Contains a `SELECT * FROM student ORDER BY student_id ASC` query and defines the PostgreSQL function `get_student_grade(p_student_id INT)` which returns a table of `(student_id, student_name, grade)` by joining the student and enrolment tables. Called from `StudentDao.getStudentGrade()`.

6.2 course.sql (Version 1 — Basic Procedure)

Contains a `SELECT * FROM course ORDER BY course_id ASC` query and the first version of the stored procedure `insert_all_data(...)` accepting 9 parameters (`DOUBLE PRECISION` for credits). Inserts one row into each of the three tables: student, course, and enrolment. No duplicate checking.

6.3 enrolment.sql (Version 2 — Production Procedure)

Contains a `SELECT * FROM enrolment ORDER BY enrolment_id ASC` query and an improved version of `insert_all_data(...)` that adds pre-insert validation using `IF EXISTS` checks. Raises an exception if the `student_id` or `course_id` already exists before inserting, preventing duplicate key violations. Uses `NUMERIC` for credits and `TEXT` types. Also wraps errors with `RAISE NOTICE` and re-raises the exception for transaction rollback.

Recommendation for juniors: Always run `enrolment.sql` — it is the production-ready version with duplicate protection.

7. Key JDBC Concepts Demonstrated

JDBC Concept	Used In	Purpose
<code>DriverManager.getConnection()</code>	All DAO classes	Opens DB connection
<code>PreparedStatement</code>	<code>StudentDao</code> , <code>CourseDao</code> , etc.	Parameterised SQL queries
<code>executeBatch()</code>	<code>StudentDao</code> , <code>CourseDao</code> , <code>EnrollmentDao</code>	Bulk insert performance
<code>CallableStatement</code>	<code>StudentDao.assignGrade()</code>	Calls stored DB function
<code>ResultSet</code>	All select methods	Iterates query results
<code>Stored Procedure</code>	<code>AllInsertDao</code>	<code>insert_all_data()</code> — 9 params
<code>DB Function</code>	<code>StudentDao</code>	<code>get_student_grade(student_id)</code>
<code>con.close()</code>	All DAO classes	Releases DB connection

7.1 JDBC Connection Flow

Every DAO class follows this pattern:

```
1.
2. Class.forName("org.postgresql.Driver");
3. Connection con = DriverManager.getConnection(url, user, pass);
4. PreparedStatement ps = con.prepareStatement(sql);
5. ps.setXxx(index, value); // bind parameters
6. ps.executeQuery() / executeUpdate() / executeBatch();
7. ResultSet rs = ... → while(rs.next()) { ... }
8. con.close();
```

8. How to Run the Project

Step 1 — Prerequisites

- Java JDK 8 or higher installed
- PostgreSQL 14+ installed and running
- Maven 3.x installed
- Eclipse IDE (or any Java IDE)

Step 2 — Create the Database

```
CREATE DATABASE "StudentManagmentSystem";
```

Then connect and create the three tables (student, course, enrolment) with the columns described in Section 4.

Step 3 — Run the SQL Files

Open pgAdmin or psql and execute student.sql, course.sql (or enrolment.sql for the production stored procedure), in that order. This creates the get_student_grade() function and the insert_all_data() stored procedure.

Step 4 — Update DB Credentials

All DAO files use hardcoded credentials. Update the password field if your local PostgreSQL password differs from the default:

```
DriverManager.getConnection(  
    "jdbc:postgresql://localhost:5432/StudentManagmentSystem",  
    "postgres",    // username  
    "root"         // change this to your password  
);
```

Step 5 — Build and Run

```
# Maven build  
mvn clean package  
  
# Run from Eclipse:  
  
Right-click StudentDriver.java → Run As → Java Application
```

8.1 Common Issues

Error	Cause	Fix
Connection refused	PostgreSQL not running	Start the PostgreSQL service
password authentication	Wrong password in DAO	Update 'root' to your actual password
relation does not exist	Tables not created	Run the CREATE TABLE SQL first
procedure does not exist	SQL files not executed	Run all 3 .sql files in pgAdmin
ClassNotFoundException	JDBC driver missing in build	Run mvn clean install to resolve

9. Application Menu Flow

Main Menu

- └─ 1. Student Menu
 - └─ 1. Register Student (batch insert)
 - └─ 2. Update Student Profile
 - └─ 3. Enroll in a Course (batch insert)
 - └─ 4. View all Courses
 - └─ 5. View all Students
 - └─ 6. View Grade (DB function call)
 - └─ 7. Go Back
- └─ 2. Admin Menu
 - └─ 1. Manage Courses (add / update / delete)
 - └─ 2. View all Students
 - └─ 3. View all Courses
 - └─ 4. View all Enrollments
 - └─ 5. Assign Grade (CallableStatement)
 - └─ 6. Insert All (stored procedure)
 - └─ 7. Go Back
- └─ 3. System Menu
 - └─ 1. Insert via Stored Procedure
 - └─ 2. Batch Execution (Course)
 - └─ 3. Return
- └─ 4. Exit

10. Learning Outcomes

- JDBC connection lifecycle management (open, use, close)
- Parameterised queries with PreparedStatement — prevents SQL injection
- Efficient bulk inserts using JDBC Batch Processing
- Invoking PostgreSQL stored procedures via PreparedStatement (CALL ...)
- Invoking PostgreSQL functions via CallableStatement
- Reading tabular results with ResultSet
- Clean layered architecture: Entity / DAO / Service / Main
- Maven dependency management for JDBC drivers
- Git & GitHub project version control

11. Future Enhancements

- GUI using Java Swing or JavaFX
- REST API layer using Spring Boot
- User authentication and role-based access control
- Web-based UI (React / Angular front-end)
- Connection pooling using HikariCP
- Unit testing with JUnit 5 and Mockito
- Docker containerisation for PostgreSQL

— End of Report —

Chetan Kailas Banait

MCA Student | Java Developer